

Implementationsdokumentation

Audio I/O & 3D-Raumdaten

Kais

6. Juli 2026

Inhaltsverzeichnis

1	Einführung	4
1.1	Architektur-Übersicht	4
2	Audio I/O Handling	5
2.1	Anforderungen (Akzeptanzkriterien)	5
2.2	Theoretischer Hintergrund	5
2.2.1	WAV-Format und Audio-Digitalisierung	5
2.2.2	WAV-Format: Technische Spezifikation	5
2.2.3	Clipping und Normalisierung	6
2.3	Implementierung: <code>wav_handler.py</code>	6
2.3.1	WAV-Einlesung	6
2.3.2	WAV-Export mit Normalisierung	6
2.3.3	Integration in die Pipeline	7
3	3D-Raumdaten: OBJ-Parser	8
3.1	Theoretischer Hintergrund	8
3.1.1	OBJ-Format	8
3.1.2	Dreiecks-Repräsentation	8
3.2	Implementierung: <code>obj_parser.py</code>	8
3.2.1	OBJ-Loader	8
3.2.2	Material-basierter Export: <code>export_room_geometry()</code>	9
3.2.3	Ausgabeformat: <code>room_geometry.json</code>	9
3.3	Blender-Workflow: Raum erstellen und exportieren	10
4	Integration und Testing	11
4.1	End-to-End-Pipeline	11
4.2	Material-Konfiguration	11
4.3	Konfiguration: <code>input_config.json</code>	12
4.4	Abhängigkeiten	12
5	Visualisierung: <code>visualizer.py</code>	13
5.1	Übersicht	13
5.2	Raumwände-Rendering	13
5.3	Dargestellte Elemente	14
5.4	Starten des Visualizers	14
6	Akzeptanzkriterien und Validierung	15
6.1	Ticket 3: Audio I/O	15
6.2	Raumdaten: OBJ-Parser & Material-Integration	15
6.3	Materialvergleich: Akustisches Verhalten	15
6.4	Abschnittsabgrenzung zur DSP-Dokumentation	16

7 Zusammenfassung und Ausblick	17
7.1 Implementierte Features	17
7.2 Mögliche Erweiterungen	17
A Code-Referenz	18
A.1 wav_handler.py	18
A.2 obj_parser.py	18
A.3 main.py	18

Kapitel 1

Einführung

Dieses Dokument beschreibt die Implementierung zweier kritischer Komponenten des Distributed Acoustic Ray Tracers:

1. **Audio I/O (Ticket 3):** Einlesen trockener WAV-Dateien und Export des verarbeiteten, halligen Audio-Signals mit Clipping-Prävention
2. **Raumdaten (OBJ-Parser):** Laden von 3D-Modellen aus `.obj`-Dateien und Konvertierung in das interne JSON-Format

Die Implementierung erfolgt in Python und integriert sich nahtlos in die bestehende Pipeline, die Rust-basierte Physik-Engine, DSP-Convolution und Distributed-Computing-Frameworks verbindet.

1.1 Architektur-Übersicht

Das System folgt einer erweiterten Pipeline mit automatischer Raumgeometrie-Integration:

- [0/4] OBJ-Import: `room.obj` → `room_geometry.json` mit Material-Absorption
- [1/4] Rust Physics Engine: Lädt Raumgeometrie, simuliert Schallstrahlen, erzeugt IR
- [2/4] Audio-Einlesung: Lädt trockene WAV-Datei
- [3/4] DSP-Convolution: Wendet IR mit frequenzabhängiger Material-Dämpfung an
- [4/4] Audio-Export: Normalisiert und speichert Ergebnis

Das Wandmaterial wird in `input_config.json` konfiguriert und durchgehend durch alle Stufen propagiert: von der Geometrie-Absorption im Rust-Tracer bis zur oktavbandbasierten Frequenzformung in der Convolution.

Kapitel 2

Audio I/O Handling

2.1 Anforderungen (Akzeptanzkriterien)

1. Sauberes Einlesen der `unprocessed.wav` ohne Qualitätsverlust
2. Export der finalen WAV-Datei ohne Artefakte
3. Normalisierung, damit geschichtetes Audio beim Export nicht clippt

2.2 Theoretischer Hintergrund

2.2.1 WAV-Format und Audio-Digitalisierung

Das WAV-Format speichert Audiodaten als zeitdiskrete Samples mit konstanter Abtastfrequenz f_s (typisch 44.1 kHz oder 48 kHz):

$$x[n] = \int_{(n-0.5)/f_s}^{(n+0.5)/f_s} x_{\text{analog}}(t) dt$$

Jedes Sample liegt im Bereich $[-1.0, 1.0]$ (bei normalisierten 16/32-Bit PCM).

2.2.2 WAV-Format: Technische Spezifikation

Das WAV-Format (Waveform Audio File Format) speichert unkomprimierte PCM-Audiodaten in einem RIFF-Container. Die für die Pipeline relevanten Parameter:

Parameter	Typische Werte	Bedeutung
Abtastrate f_s	44.100 Hz, 48.000 Hz	Samples pro Sekunde
Bit-Tiefe	16-bit, 24-bit, 32-bit float	Amplitude-Auflösung
Kanäle	1 (Mono), 2 (Stereo)	Anzahl unabhängiger Signale
Kodierung	PCM (Integer) oder IEEE Float	Speicherformat der Samples

Tabelle 2.1: WAV-Formatparameter und ihre Bedeutung für die Pipeline

Die `soundfile`-Bibliothek normalisiert beim Einlesen automatisch: 16-bit-Integer-Samples im Bereich $[-32768, 32767]$ werden in `float64`-Werte im Bereich $[-1.0, 1.0]$ umgewandelt. Diese Skalierung ist verlustfrei – die Quantisierungsschritte des PCM-Formats bleiben erhalten. 32-bit-Float-WAV wird direkt übernommen. Die Pipeline erwartet stets Float-Daten; `write_wav` schreibt standardmäßig 32-bit-Float zurück, was für Hall-Anwendungen bei $f_s = 44.100$ Hz ausreicht.

Abtastraten-Konsistenz: Die Rust-Engine ist in `input_config.json` mit einer festen Abtastrate konfiguriert (Standard: 44.100 Hz). Stimmt die Abtastrate der Eingangs-WAV nicht

überein, gibt die Pipeline eine Warnung aus ohne automatisches Resampling, da eine Rateninkonsistenz alle Verzögerungen der Impulsantwort um den Ratenquotienten staucht oder dehnt und den Hall hörbar verstimmt.

2.2.3 Clipping und Normalisierung

Wenn während der Convolution (Faltung) die Amplitude $|y[n]|$ über 1.0 steigt, tritt **Clipping** auf:

$$y_{\text{clipped}}[n] = \begin{cases} 1.0 & \text{wenn } y[n] > 1.0 \\ y[n] & \text{wenn } |y[n]| \leq 1.0 \\ -1.0 & \text{wenn } y[n] < -1.0 \end{cases}$$

Um dies zu vermeiden, normalisieren wir:

$$y_{\text{norm}}[n] = \frac{y[n]}{A_{\text{max}}} \quad \text{wobei} \quad A_{\text{max}} = \max_n |y[n]|$$

Diese Normalisierung ist bewusst **konditional**: Liegt das Maximum bereits unter 1.0, wird nicht skaliert – leise Signale werden nicht künstlich lauter. Die Pegelrelation zwischen verschiedenen Material-Läufen bleibt damit erhalten, was akustische Vergleiche ermöglicht.

2.3 Implementierung: wav_handler.py

2.3.1 WAV-Einlesung

```

1 import numpy as np
2 import soundfile as sf
3
4 def read_wav(wav_path: str) -> tuple:
5     """
6     Reads a WAV file and returns the audio data as a NumPy array
7     along with the corresponding sample rate.
8     """
9     audio_data, sample_rate = sf.read(wav_path)
10    return audio_data, sample_rate

```

Funktionsweise:

- Die soundfile-Bibliothek liest WAV-Dateien robust
- `audio_data` ist ein NumPy-Array der Form `(samples,)` oder `(samples, channels)`
- `sample_rate` ist die Abtastfrequenz in Hz

2.3.2 WAV-Export mit Normalisierung

```

1 def write_wav(wav_path: str, audio_data, sample_rate: int) -> None:
2     """
3     Normalizes the audio data to prevent clipping and exports it to a WAV file.
4     """
5     max_amplitude = np.max(np.abs(audio_data))
6     if max_amplitude > 1.0:
7         audio_data = audio_data / max_amplitude
8     sf.write(wav_path, audio_data, sample_rate)

```

Algorithmus:

Algorithm 1 Normalisierter WAV-Export

```
 $A_{\max} \leftarrow \max_n |y[n]|$   
if  $A_{\max} > 1.0$  then  
     $y[n] \leftarrow y[n]/A_{\max}$  für alle  $n$   
end if  
Schreibe normalisierte Samples in WAV-Datei
```

2.3.3 Integration in die Pipeline

```
1 # main.py Schritt [2/4]: Audio laden  
2 dry_audio, sample_rate = read_wav(input_wav_path)  
3  
4 # Schritt [3/4]: Convolution mit Material-Absorption  
5 conv_material = ir_data.get("metadata", {}).get("wall_material")  
6 wet_audio = convolve(dry_audio, sample_rate, ir_data, material=conv_material)  
7  
8 # Schritt [4/4]: Normalisieren und exportieren  
9 write_wav(output_wav_path, wet_audio, sample_rate)
```

Der `wall_material`-Wert wird automatisch aus `ir_output.json` gelesen, das der Rust-Tracer nach der Simulation schreibt. Damit nutzen Tracer und Convolution stets dasselbe Material.

Kapitel 3

3D-Raumdaten: OBJ-Parser

3.1 Theoretischer Hintergrund

3.1.1 OBJ-Format

Das Wavefront .obj-Format ist ein textbasiertes 3D-Modellformat. Jede Zeile beginnt mit einem Schlüsselwort, das den Zeilentyp bestimmt:

```
1 # Vertices (Ecken)
2 v 1.0 2.0 -1.5 # Vertex 1
3 v 2.0 2.0 -1.5 # Vertex 2
4 v 1.0 3.0 -1.5 # Vertex 3
5
6 # Faces (Dreiecke, Format: v/vt/vn)
7 f 1/1/1 2/2/1 3/3/1 # Textur- und Normal-Indizes optional
```

Der Parser verarbeitet nur die für Raytracing relevanten Zeilen:

Zeilentyp	Bedeutung	Verarbeitet?
v x y z	Vertex-Position	✓
f v1 v2 v3	Dreiecksfläche (Vertex-Indizes)	✓
f v/vt/vn	Face mit Textur- und Normalindex	✓(nur v)
vn nx ny nz	Vertex-Normale	Ignoriert
vt u v	Textur-Koordinate	Ignoriert
usemtl name	Material-Zuweisung (.mtl)	Ignoriert
# ...	Kommentar	Ignoriert

Tabelle 3.1: OBJ-Zeilentypen und Verarbeitungsstatus im Parser

Indexierung: OBJ nutzt 1-basierte Indizes, Python 0-basierte – der Parser subtrahiert beim Einlesen automatisch 1.

3.1.2 Dreiecks-Repräsentation

$$\text{Triangle} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix}$$

3.2 Implementierung: obj_parser.py

3.2.1 OBJ-Loader

```

1 def load_obj_to_triangles(file_path: str) -> list:
2     vertices = []
3     triangles = []
4     path = Path(file_path)
5     if not path.exists():
6         raise FileNotFoundError(f"The 3D file '{file_path}' was not found.")
7     with path.open('r') as f:
8         for line in f:
9             line = line.strip()
10            if line.startswith('v '):
11                parts = line.split()
12                x, y, z = float(parts[1]), float(parts[2]), float(parts[3])
13                vertices.append([x, y, z])
14            elif line.startswith('f '):
15                parts = line.split()
16                v1_idx = int(parts[1].split('/')[0]) - 1
17                v2_idx = int(parts[2].split('/')[0]) - 1
18                v3_idx = int(parts[3].split('/')[0]) - 1
19                triangle = [vertices[v1_idx], vertices[v2_idx], vertices[v3_idx]]
20                triangles.append(triangle)
21    return triangles

```

3.2.2 Material-basierter Export: export_room_geometry()

Die neue Exportfunktion verbindet OBJ-Geometrie mit der Materialdatenbank (`materials.json`) und schreibt `room_geometry.json` für den Rust-Tracer. Jedes Dreieck erhält die mittlere Absorptionskonstante des gewählten Materials:

```

1 _MATERIALS_JSON = Path(__file__).resolve().parent.parent / "rust_service" / "materials
  .json"
2
3 def _mean_absorption(material: str) -> float:
4     """Mittlere Absorption aus materials.json fuer das gewaehlte Material."""
5     data = json.loads(_MATERIALS_JSON.read_text())
6     alphas = data["materials"][material]["absorption"]
7     return sum(alphas) / len(alphas)
8
9 def export_room_geometry(obj_path: str, material: str, out_path: str) -> None:
10    triangles_raw = load_obj_to_triangles(obj_path)
11    absorption = _mean_absorption(material)
12    triangles = [
13        {"v0": t[0], "v1": t[1], "v2": t[2], "absorption": absorption}
14        for t in triangles_raw
15    ]
16    out = {"triangles": triangles, "triangle_count": len(triangles)}
17    Path(out_path).write_text(json.dumps(out, indent=2))

```

3.2.3 Ausgabeformat: room_geometry.json

Der Rust-Tracer liest Dreiecke inkl. Material-Absorption aus dieser Datei:

```

1 {
2     "triangles": [
3         {"v0": [0.0, 0.0, 0.0], "v1": [12.0, 0.0, 0.0],
4          "v2": [12.0, 12.0, 0.0], "absorption": 0.033}
5     ],

```

```
6 | "triangle_count": 12
7 | }
```

Die **absorption**-Werte entsprechen dem arithmetischen Mittel aller 8 Oktavbänder aus **materials.json**. Die frequenzabhängige Verarbeitung findet ausschließlich in der Python-Convolution statt.

3.3 Blender-Workflow: Raum erstellen und exportieren

Für realistische Raumakustik kann jede beliebige Raumgeometrie aus Blender exportiert werden. Der empfohlene Workflow:

1. **Raum modellieren:** Geschlossenen Raum in Blender erstellen. Alle Flächen müssen nach innen zeigen (*Face Normals* prüfen).
2. **Triangulieren:** *Edit Mode* → *Mesh* → *Faces* → *Triangulate Faces* (Shortcut: **Ctrl+T**). Der OBJ-Parser verarbeitet ausschließlich Dreiecke.
3. **Exportieren:** *File* → *Export* → *Wavefront (.obj)*
 - Koordinatensystem: Y Forward, Z Up (passend zu **input_config.json**)
 - Haken bei *Triangulate Mesh* setzen
 - Speichern als **room.obj** in **python_service/**
4. **Positionen prüfen:** Speaker- und Mic-Positionen in **input_config.json** müssen innerhalb des Modells liegen. Die Mic-Position zuzüglich **mic_radius** darf keine Wand überschreiten.
5. **Pipeline starten:** **python main.py** erkennt **room.obj** automatisch und generiert **room_geometry.json** vor dem Rust-Lauf.

Kapitel 4

Integration und Testing

4.1 End-to-End-Pipeline

Die Pipeline verläuft nun in 5 Schritten. Schritt 0 ist vollautomatisch: `room.obj` wird erkannt, mit dem konfigurierten Material in `room_geometry.json` umgewandelt und dem Rust-Tracer bereitgestellt.

```
1 # Schritt 0: OBJ -> room_geometry.json (automatisch wenn room.obj vorhanden)
2 if Path("room.obj").exists():
3     export_room_geometry("room.obj", wall_material,
4                           "../rust_service/room_geometry.json")
5
6 # Schritt 1: Rust Physics Engine
7 ir_data = run_rust_and_load_json(rust_binary_path, json_path)
8
9 # Schritt 2: Audio laden
10 dry_audio, sample_rate = read_wav(input_wav_path)
11
12 # Schritt 3: Convolution mit Material-Absorption
13 conv_material = ir_data.get("metadata", {}).get("wall_material")
14 wet_audio = convolve(dry_audio, sample_rate, ir_data, material=conv_material)
15
16 # Schritt 4: Normalisieren und exportieren
17 write_wav(output_wav_path, wet_audio, sample_rate)
```

4.2 Material-Konfiguration

Das Wandmaterial wird in `rust_service/input_config.json` festgelegt:

```
1 {
2     "wall_material": "concrete",
3     "max_bounces": 100,
4     "rays_to_cast": 1000
5 }
```

Vom Rust-Tracer wird `wall_material` in `ir_output.json` exportiert und von der Python-Pipeline automatisch für die Convolution übernommen. Vegerfügbare Materialien (aus `materials.json`):

Material	Beschreibung	Mittl. Absorption
concrete	Beton (stark reflektierend)	0.033
drywall	Trockenbau (mäßig)	0.150
glass	Glas (stark reflektierend)	0.026
carpet	Teppich (stark absorbierend)	0.425
foam	Schaum (sehr stark absorbierend)	0.650

Tabelle 4.1: Verfügbare Materialien in `materials.json`

4.3 Konfiguration: `input_config.json`

Alle Simulationsparameter werden in einer einzigen JSON-Datei gebündelt, die sowohl vom Rust-Tracer als auch von der Python-Pipeline gelesen wird:

Feld	Beispielwert	Bedeutung
<code>speaker_position</code>	[2.0, 3.0, 1.5]	3D-Position der Schallquelle
<code>mic_position</code>	[8.0, 7.0, 6.0]	3D-Position des Mikrofons
<code>mic_radius</code>	3.0	Einfangradius des Mikrofons in Metern
<code>rays_to_cast</code>	1000	Anzahl der simulierten Strahlen
<code>max_bounces</code>	100	Max. Reflexionen pro Strahl
<code>min_pressure</code>	0.001	Abbruchschwelle (Restdruck)
<code>wall_material</code>	"concrete"	Wandmaterial (aus <code>materials.json</code>)

Tabelle 4.2: Felder der `input_config.json`

Das Feld `wall_material` wird von der Python-Pipeline beim Start ausgelesen, um `room_geometry.json` mit der passenden Absorption zu generieren. Der Rust-Tracer schreibt es anschließend in `ir_output.json`, von wo die Convolution es für die frequenzabhängige Dämpfung übernimmt.

4.4 Abhängigkeiten

Modul	Bibliotheken	Version
<code>wav_handler.py</code>	soundfile, numpy	Any
<code>obj_parser.py</code>	pathlib, json	Built-in
<code>convolution.py</code>	scipy, numpy	Any
<code>materials.json</code>	— (Datenbasis)	—

Kapitel 5

Visualisierung: visualizer.py

5.1 Übersicht

Der Visualizer rendert die Ergebnisse der Rust-Simulation als interaktive 3D-Szene im Browser (Plotly). Er liest zwei Dateien aus dem `rust_service`:

- `visualisation_data.json`: Strahlpfade, Speaker- und Mic-Position
- `room_geometry.json`: Raumwände als Dreiecksliste (optional)

5.2 Raumwände-Rendering

Die Funktion `_draw_room_walls()` wurde neu implementiert und lädt `room_geometry.json`, falls vorhanden, und zeichnet jedes Dreieck als blaues Drahtgitter:

```
1 def _draw_room_walls(fig: go.Figure, geometry_path: Path) -> None:
2     if not geometry_path.exists():
3         return
4     data = json.loads(geometry_path.read_text())
5     for tri in data["triangles"]:
6         v0, v1, v2 = tri["v0"], tri["v1"], tri["v2"]
7         xs = [v0[0], v1[0], v2[0], v0[0]]
8         ys = [v0[1], v1[1], v2[1], v0[1]]
9         zs = [v0[2], v1[2], v2[2], v0[2]]
10        fig.add_trace(go.Scatter3d(
11            x=xs, y=ys, z=zs, mode='lines',
12            line=dict(width=1, color='rgba(100,130,200,0.35)'),
13            showlegend=False, hoverinfo='skip'))
```

Durch das Rückschließen des Dreiecks (`v0` als vierter Punkt) entsteht ein geschlossenes Dreieck. Die halbtransparente Farbe `rgba(100,130,200,0.35)` hält die Raumwände visuell im Hintergrund, sodass die Strahlpfade im Vordergrund bleiben.

5.3 Dargestellte Elemente

Element	Darstellung	Datenquelle
Strahlpfade	Farbige Linien (Golden-Angle-Farbfolge)	visualisation_data.json
Lautsprecher	Blauer Kreuz-Marker	visualisation_data.json
Mikrofon-Zentrum	Blauer Punkt	visualisation_data.json
Mikrofon-Radius	Rote Kugel (halbtransparent)	visualisation_data.json
Raumwände	Hellblaues Drahtgitter	room_geometry.json

Tabelle 5.1: Visuelle Elemente der 3D-Debugging-Szene

Die Golden-Angle-Farbfolge $\varphi_i = (i \cdot 137,5\%) \bmod 360$ verteilt die Strahlfarben gleichmäßig im HSL-Farbraum, sodass auch bei vielen Strahlen kein Farbton doppelt vorkommt.

5.4 Starten des Visualizers

```

1 cd python_service
2 python visualizer.py

```

Plotly öffnet automatisch einen Browser-Tab mit der interaktiven 3D-Szene. Fehlt `room_geometry.json`, werden keine Raumwände gezeichnet; die Datei wird durch Ausführen der Pipeline mit einer `room.obj` erzeugt.

Kapitel 6

Akzeptanzkriterien und Validierung

6.1 Ticket 3: Audio I/O

Kriterium	Erfüllt?	Test
Sauberes Einlesen .wav	✓	<code>len(audio_data) > 0</code>
Export ohne Qualitätsverlust	✓	Hörtest, Spektrum-Analyse
Normalisierung gegen Clipping	✓	<code>max(abs(audio)) <= 1.0</code>

6.2 Raumdaten: OBJ-Parser & Material-Integration

Kriterium	Erfüllt?	Test
Parser liest .obj	✓	<code>len(triangles) > 0</code>
JSON-Export mit Absorption	✓	absorption-Feld pro Dreieck
Rust lädt OBJ-Geometrie	✓	room_geometry.json erkannt
Material durchgereicht	✓	wall_material in ir_output.json
Frequenzabhäng. Convolution	✓	apply_material_absorption()

6.3 Materialvergleich: Akustisches Verhalten

Die folgende Tabelle zeigt das Verhalten der Pipeline mit dem Test-Raum (12 m × 12 m × 10 m, 100 Strahlen, 100 maximale Bounces):

Material	Hits	Eff. Bounces	Klangcharakter
concrete	~2000	bis 100	Hell, langer Nachhall (Kathedrale)
glass	~2200	bis 100	Sehr hell, extrem langer Nachhall
drywall	~1500	~43	Moderat, typischer Wohnraum
carpet	~400	~11	Dumpf, kurzer Nachhall
foam	~200	~7	Sehr trocken, fast kein Nachhall

Tabelle 6.1: Verhalten verschiedener Materialien (Richtwerte, 12 m × 12 m × 10 m Testraum)

Die **effektiven Bounces** ergeben sich aus der Bedingung $(1 - \alpha)^n < p_{\min}$ mit $p_{\min} = 0.001$: $n < \ln(0.001)/\ln(1 - \alpha)$. Bei Beton ($\alpha = 0.033$) benötigt ein Strahl theoretisch 204 Bounces, wird aber durch `max_bounces = 100` begrenzt. Bei Teppich ($\alpha = 0.425$) bricht er bereits nach 11 Bounces ab.

6.4 Abschnittsabgrenzung zur DSP-Dokumentation

Die mathematischen Details der Impulsantwort-Konstruktion (`build_impulse_response`), die Wahl des Faltungsalgorithmus (`oaconvolve`) sowie die vollständige Oktavband-Filterbank sind in Kapitel 5 (Signalverarbeitung, Cem Acar) dokumentiert. Der vorliegende Abschnitt beschränkt sich auf die Schnittstellenperspektive: Was `convolve()` als Vertrag anbietet und wie es in die Gesamtpipeline eingebunden ist.

Kapitel 7

Zusammenfassung und Ausblick

7.1 Implementierte Features

1. **Audio I/O:** Robust WAV-Lesen/Schreiben mit `soundfile`
2. **Clipping-Prävention:** Peak-Normalisierung vor Export
3. **OBJ-Parser:** Liest Vertices/Faces, konvertiert in Dreiecke
4. **Material-Integration:** `export_room_geometry()` verbindet OBJ mit `materials.json`
5. **Rust-OBJ-Pipeline:** Rust lädt `room_geometry.json` automatisch
6. **Material-Durchleitung:** `wall_material` von Config bis Convolution
7. **Frequenzabhängige Dämpfung:** Oktavband-EQ via `apply_material_absorption()`
8. **Pipeline-Integration:** Vollautomatisch ab Schritt 0 (OBJ-Erkennung)

7.2 Mögliche Erweiterungen

1. **Multi-Channel Audio:** Stereo/5.1 Surround Support
2. **Per-Dreieck-Materialien:** Verschiedene Materialien pro Wandfläche aus `.mtl`-Datei
3. **Adaptive Normalisierung:** Loudness-basierte Pegel (LUFS) statt Peak
4. **Geometrie-Optimierungen:** BVH-Tree für schnelle OBJ-Intersects

Anhang A

Code-Referenz

A.1 wav_handler.py

Code-Referenz: `python_service/wav_handler.py`

A.2 obj_parser.py

Code-Referenz: `python_service/obj_parser.py`

A.3 main.py

Code-Referenz: `python_service/main.py`